1 Лекция 02. Ограниченные деревья поиска и анализ ветвления

1.1 Что такое branching и какую задачу мы решаем

Ветвление (branching) — способ построить точный алгоритм: вместо одного неизвестного решения рассматривают несколько исчерпывающих вариантов следующего выбора. Каждый вариант порождает меньшую задачу того же типа, которую решают рекурсивно. Результат YES получается, если успешен хотя бы один вариант. Никакого случайного угадывания здесь нет: алгоритм детерминированно проверяет все необходимые случаи.

Само слово branching не обозначает отдельную вычислительную задачу. Это метод. Его основной пример здесь — **Vertex Cover**, задача о вершинном покрытии. На входе конечный простой неориентированный граф $G = (V, E)$ и целое $k \geq 0$. Требуется найти множество $S \subseteq V$ размера не больше k, такое, что для каждого ребра uv выполнено $u \in S$ или $v \in S$, либо доказать, что такого множества нет.

Например, для треугольника abc одной вершины недостаточно: ребро между двумя другими останется непокрытым. Любые две вершины покрывают все рёбра. Поэтому $(K_3, 1)$ — NO, $(K_3, 2)$ — YES. Покрытие не обязано быть независимым: выбирать оба конца ребра разрешено.

Цель главы — понять, как превратить такое разбиение на случаи в FPT-алгоритм и как оценить его время точнее, чем числом всех подмножеств графа. Нужны определения FPT из первой главы, рекурсия и индукция; все графовые обозначения вводятся ниже.

1.2 От перебора подмножеств к дереву поиска

Перебор всех подмножеств до k вершин даёт примерно n^k кандидатов. В нём много ненужных вариантов: кандидат может не покрывать даже заранее известное ребро. Ветвление использует обязательное локальное условие.

Выберем ребро uv. Любое покрытие содержит u или v; третий вариант, не выбирать никого, сразу невозможен. Проверим:

1. Включить u в решение и покрыть оставшиеся рёбра не более чем $k - 1$ вершинами.
2. Включить v и решить аналогичный остаток.

Оба случая могут быть верными одновременно. Для вопроса существования это нормально: логическое OR не требует непересекающихся случаев. При подсчёте числа решений такая схема требует исправления, иначе покрытия, содержащие обоих, могут учитываться дважды.

Дерево поиска (search tree) — дерево рекурсивных вызовов. Корень соответствует исходной задаче, дети — вариантам первого выбора, листья — вызовам, в которых ответ уже известен. Ограниченное дерево поиска (bounded search tree) в FPT-анализе имеет число узлов, ограниченное функцией параметра, с допустимым полиномиальным множителем для обработки данных.

1.3 Состояние рекурсии и смысл удаления

Пусть A — множество уже выбранных вершин, $H = G - A$ — остаточный граф, $\ell = k - |A|$ — остаточный бюджет. Обозначение $G - A$ означает удаление вершин A вместе со всеми инцидентными рёбрами. Эти рёбра уже покрыты выбранными вершинами. Новая задача — найти покрытие H размера не больше ℓ.

Таков **инвариант**: решения остатка при добавлении A становятся решениями исходной задачи, согласованными со сделанными выборами. Можно передавать A явно, а можно возвращать из рекурсии остаточное покрытие и добавлять выбранную вершину при обратном ходе.

Следующий псевдокод возвращает множество или специальное значение FAIL, отличное от пустого множества:

1. Если $\ell < 0$, вернуть FAIL.
2. Если $E(H) = \varnothing$, вернуть $\varnothing$.
3. Выбрать любое ребро $uv \in E(H)$.
4. Вызвать алгоритм для $(H - u, \ell - 1)$. Если получено множество R, вернуть $R \cup \{u\}$.
5. Вызвать алгоритм для $(H - v, \ell - 1)$. Если получено R, вернуть $R \cup \{v\}$.
6. Если оба вызова неуспешны, вернуть FAIL.

Отрицательный бюджет проверяется первым: пустое множество имеет размер 0, а значит, не укладывается в бюджет -1. Изолированные вершины можно игнорировать, поскольку они не покрывают никаких рёбер. Когда рёбер нет, не нужно расходовать оставшийся бюджет полностью: требуется «не больше», а не «ровно».

1.4 Полное доказательство корректности

Обозначим через $\mathrm{VC}(H, \ell)$ утверждение о существовании покрытия размера не больше ℓ. Для ребра uv:

$$\mathrm{VC}(H, \ell) \iff \mathrm{VC}(H - u, \ell - 1) \vee \mathrm{VC}(H - v, \ell - 1).$$

Слева направо. Пусть S — покрытие H. Если $u \in S$, то $S \setminus \{u\}$ покрывает $H - u$: каждое оставшееся ребро уже покрывалось каким-то концом, отличным от u. Размер уменьшился на один. Если $u \notin S$, обязательно $v \in S$, и подходит вторая ветвь.

Справа налево. Если R покрывает $H - u$, то $R \cup \{u\}$ покрывает H: рёбра с концом u покрыты u, а остальные — R. Размер не больше ℓ. Для v аргумент тот же.

Базы псевдокода правильны непосредственно. В остальных вызовах бюджет уменьшается на один, поэтому рекурсия заканчивается. Индукцией по $\ell \geq 0$, используя доказанную эквивалентность и базу отрицательного бюджета, получаем: алгоритм возвращает покрытие нужного размера тогда и только тогда, когда оно существует.

Это две разные проверки. Уменьшение бюджета доказывает остановку; эквивалентность доказывает, что остановившийся алгоритм даёт правильный ответ.

1.5 Пример полного поиска

Возьмём путь $a - b - c - d$ с бюджетом 1 и выберем ребро ab. В ветви выбора a останутся рёбра bc, cd с бюджетом 0. Какой бы конец следующего ребра мы ни выбрали, бюджет станет отрицательным; ветвь неуспешна. В ветви выбора b останется ребро cd с бюджетом 0 — тоже неуспех. Значит, ответ NO.

На том же графе с бюджетом 2 после выбора b получаем одно ребро cd и бюджет 1. Выбрав c, получаем граф без рёбер и бюджет 0. При обратном ходе возвращаются $\varnothing$, затем $\{c\}$, затем $\{b, c\}$. Алгоритм нашёл покрытие, не перебирая все пары вершин.

1.6 Почему получается FPT

Вызов на глубине i имеет бюджет $k-i$. Глубина не больше $k+1$, поскольку отрицательный бюджет немедленно завершает вызов. На глубине i не больше 2^i узлов, поэтому полное дерево имеет меньше 2^{k+2} узлов. Ранний выход после найденного решения только уменьшает работу.

При прямой реализации граф в каждом узле копируется и просматривается за $O(n+m)$, где n, m относятся к исходному графу. Отсюда время $O(2^k(n+m))$. При обходе в глубину не обязательно хранить всё дерево одновременно: достаточно текущего стека вызовов и информации для восстановления.

Ключевое сочетание — ограниченное число вариантов и уменьшение бюджета в каждом из них. Две ветви глубины n дали бы 2^n, а до n ветвей глубины k — $n^{O(k)}$. Сам факт рекурсивного перебора ещё не означает FPT.

1.7 Неравные ветви и ветвящий вектор

Иногда за один выбор платим разное число единиц бюджета. Если в r ветвях мера ℓ уменьшается хотя бы на положительные целые константы $a_1, \dots, a_r$, говорят о **ветвящем векторе (branching vector)** $(a_1, \dots, a_r)$. Мера — величина, которой мы контролируем глубину; пока это остаточный бюджет. В более сложных алгоритмах она может отличаться от него, и тогда нужно отдельно ограничить начальную меру функцией параметра.

Для верхней оценки числа листьев используем рекуррентную мажоранту

$$L(\ell) \le \sum_{i=1}^{r} L(\ell - a_i).$$

Для $r \ge 2$ определим **ветвящее число (branching number)** $\rho > 1$ из уравнения

$$\sum_{i=1}^{r} \rho^{-a_i} = 1.$$

Левая часть непрерывна, строго убывает, в точке 1 равна $r > 1$, а на бесконечности стремится к 0. Поэтому нужный корень единственен. При векторе $(1,1)$ он равен 2, а при $(1,2)$ уравнение $\rho^{-1} + \rho^{-2} = 1$ равносильно $\rho^2 = \rho + 1$, откуда $\rho = (1+\sqrt{5})/2$.

Доказательство экспоненциальной оценки. Пусть $a = \max a_i$. Для бюджетов от $1-a$ до 0 зададим постоянные базы, покрывающие терминальные случаи. Выберем C так, чтобы $L(j) \le C\rho^j$ на этом конечном диапазоне. Тогда по индукции

$$L(\ell) \le \sum_i C\rho^{\ell - a_i} = C\rho^\ell \sum_i \rho^{-a_i} = C\rho^\ell.$$

Если терминальный случай возникает при положительном бюджете, увеличим C до как минимум 1. Для дерева с I внутренними узлами и L листьями число рёбер равно $I + L - 1$ и не меньше $2I$. Следовательно, $I \le L - 1$, всего узлов не больше $2L - 1$. Так листовая оценка даёт время $O^*(\rho^k)$. Цепочки преобразований с одним ребёнком этим аргументом не учитываются: их длину нужно ограничить отдельно.

1.8 Более сильный выбор для Vertex Cover

Для вершины v обозначим её соседей через $N_H(v)$, замкнутую окрестность через $N_H[v] = N_H(v) \cup \{v\}$, степень через $d = |N_H(v)|$. Всякое покрытие либо содержит v, либо не содержит её и тогда обязано содержать **всех** соседей: иначе какое-то ребро vu не будет покрыто.

Получаем две остаточные задачи:

$$(H - v, \ell - 1), \qquad (H - N_H[v], \ell - d).$$

В первой ветви выбираем v. Во второй выбираем $N_H(v)$; саму v удаляем бесплатно, поскольку после удаления всех соседей она изолирована. К любому покрытию второго остатка можно добавить всех соседей, покрыв и рёбра между соседями, и их рёбра наружу, и рёбра к v.

Полнота следует из разбора «v выбрана / не выбрана». Первая ветвь может дать покрытие, содержащее также всех соседей; непересекаемость для решения вопроса существования по-прежнему не нужна. Вектор равен $(1, d)$.

1.9 Почему графы степени не больше 2 лучше решить сразу

Если максимальная степень не больше 2, каждая связная компонента — отдельная вершина, путь или цикл. Обоснование: если в компоненте есть вершина степени 1, идём от неё, не возвращаясь назад; ветвиться нельзя, поэтому получаем путь, охватывающий компоненту. Если все степени равны 2, такое движение замыкается в цикл, и присоединить к нему другие вершины при этой границе степеней невозможно.

Путь на r вершинах имеет минимальное покрытие размера $\lfloor r/2 \rfloor$. Выбор вершин с чётными номерами достигает этой границы. С другой стороны, рёбра $(1, 2), (3, 4), \ldots$ попарно не пересекаются; каждому нужен свой выбранный конец. Такой набор непересекающихся рёбер называется **паросочетанием (matching)**.

Цикл на $r \geq 3$ вершинах имеет минимум $\lceil r/2 \rceil$. Нижняя оценка: одна вершина покрывает только два ребра, а рёбер r. Для чётного цикла берём каждую вторую вершину; для нечётного — вершины $2, 4, \ldots, r-1$ и r. Это покрытие нужного размера. Изолированная вершина даёт вклад 0.

Минимумы компонент складываются, потому что между компонентами нет рёбер. За полиномиальное время находим их, строим покрытия и сравниваем суммарный размер с бюджетом.

1.10 Алгоритм с оценкой $O^*(1.4656^k)$

Сначала обрабатываем отрицательный бюджет. Если максимальная степень не больше 2, используем только что описанный точный алгоритм. Иначе выбираем вершину степени $d \geq 3$ и применяем две ветви предыдущего раздела. Вторая уменьшает бюджет хотя бы на 3, поэтому для оценки достаточно вектора $(1, 3)$:

$$\rho^{-1} + \rho^{-3} = 1, \qquad \rho^3 = \rho^2 + 1, \qquad \rho \approx 1.465571232.$$

Если $d > \ell$, вторую ветвь можно сразу считать неуспешным листом. Чтобы формально применять постоянные базы, мажорируем её фиктивной ветвью с уменьшением ровно на 3: более сильное уменьшение и ранняя остановка не увеличивают размер дерева. Для $\ell = 0, 1, 2$ число узлов ограничено абсолютной константой. Получаем $O^*(1.4656^k)$ с полиномиальной работой в каждом узле. Это учебная оценка, достаточная для демонстрации метода.

Корректность улучшенного алгоритма состоит из уже доказанной эквивалентности ветвей и точного решения базового класса графов. Для восстановления во второй ветви добавляем всех выбранных соседей, в первой — вершину v.

1.11 Перенос на d-Hitting Set

Даны конечная вселенная U, семейство подмножеств $\mathcal{F}$ и бюджет k. **Пересекающее множество (hitting set)** — $X \subseteq U$, для которого $X \cap F \neq \varnothing$ для каждого $F \in \mathcal{F}$. В d-Hitting Set каждое требование имеет размер не больше d. Vertex Cover — частный случай $d = 2$: каждое ребро задаёт двухэлементное требование.

Если $\varnothing \in \mathcal{F}$, ответ NO: пересечь пустое множество невозможно. Если семейство пусто и бюджет неотрицателен, ответ YES. Иначе выбираем любое $F \in \mathcal{F}$ и делаем по ветви для каждого $a \in F$: выбираем a, удаляем **все требования, содержащие** a, и уменьшаем бюджет на 1. Остальные требования сохраняются. Любое решение выбирает хотя бы один элемент F, а добавление выбранного элемента к решению остатка удовлетворяет удалённые требования. Это доказывает обе стороны корректности.

Ветвей не больше d, глубина ограничена $k + 1$, получаем $O^*(d^k)$ для $d \geq 2$. При $d = 1$ задача решается непосредственно выбором всех элементов одноэлементных требований. При фиксированном d это FPT по k; при переменном d — по совместному параметру (k, d).

1.12 Когда найденного препятствия недостаточно

В **Feedback Vertex Set** нужно удалить не больше k вершин так, чтобы не осталось циклов. Любой найденный цикл должен потерять хотя бы одну вершину. Однако цикл может иметь длину n даже при $k = 1$. Ветвление по всем его вершинам не имеет числа детей, ограниченного функцией k, и даёт только грубую XP-оценку $n^{O(k)}$. Для FPT здесь нужны дополнительные идеи.

При разработке нового ветвления проверяйте: почему каждый ответ попадает хотя бы в одну ветвь; почему решение каждой ветви восстанавливается; что убывает; чем ограничено начальное значение меры; сколько работы требуется на узел. Эти вопросы превращают интуицию «попробуем все случаи» в доказанный алгоритм.

1.13 Самопроверка

Объясните без формул, зачем во второй ветви удаляется также v, хотя бюджет тратится только на её соседей. Затем восстановите рекуррентную оценку и индукцию для $(1, 3)$. Наконец, объясните, почему ограничение степени проверяется в каждом остаточном графе, а не только в исходном.

1.14 Практика

Задачи 02.1–02.5. Новые задачи дают два применения ветвления вне уже разобранного Vertex Cover; решения сохранены отдельно.

1.15 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §§3.1–3.2, с. 53–56. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.